

Getting the Most out of the LLVM Auto Vectorizer for RISC-V Vectors (RVV)

Kolya Panchenko, SiFive

Different types of vectorization

Loop Vectorization	`void foo(const int32_t N, const int8_t *a, float *b) {`
	`  for (int32_t i = 0; i < N; ++i) {`
	`  b[i] = a[i] * a[i];`
	`  }`
	`}`

SLP (Superword-Level Parallelism)	`void foo_4(const int8_t *a, float *b) {`
	`  b[0] = a[0] * a[0];`
	`  b[1] = a[1] * a[1];`
	`  b[2] = a[2] * a[2];`
	`  b[3] = a[3] * a[3];`
	`}`

VLA vs VLS loop vectorization

Scalar	`void foo(const int32_t N, const int8_t *a, float *b) {`
	`  for (int32_t i = 0; i < N; ++i) {`
	`  b[i] = a[i] * a[i];`
	`  }`
	`}`

VLS (VL = 4)	`void foo(const int32_t N, const int8_t *a, float *b) {`
	`  // Main vector loop`
	`  for (int32_t i = 0; i < N; i += 4) {`
	`  b[i:i+3] = a[i:i+3] * a[i:i+3];`
	`  }`
	`  // Remainder loop`
	`  for (int32_t i = N - (N & 3); i < N; ++i) {`
	`  b[i] = a[i] * a[i];`
	`  }`
	`}`

VLA vs VLS loop vectorization (cont.)

VLA (in upstream)	`void foo(const int32_t N, const int8_t *a, float *b) {`
	`  // Main vector loop`
	`  VL = llvm.vscale.i64()`
	`  for (int32_t i = 0; i < N; i += VL) {`
	`  b[i:i+VL-1] = a[i:i+VL-1] * a[i:i+VL-1];`
	`  }`
	`  // Remainder loop`
	`  for (int32_t i = N - (N & (VL-1)); i < N; ++i) {`
	`  b[i] = a[i] * a[i];`
	`  }`
	`}`

RVV VLA	`void foo(const int32_t N, const int8_t *a, float *b) {`
	`  // Main vector loop`
	`  for (int32_t i = 0; i < N; i += VL) {`
	`  VL = vsetvli N-i, …`
	`  b[i:i+VL-1] = a[i:i+VL-1] * a[i:i+VL-1];`
	`  }`
	`}`

RVV VLA vs VLS first perf experiments*

	A	B	C	D
1		Speedup VLS vs RVV VLA LMUL=1		Code size VLS vs RVV VLA LMUL=1 (>0 is better)
2	400.perlbench	102.47%		-0.75%
3	401.bzip2	105.87%		0.21%
4	403.gcc	102.31%		-1.48%
5	429.mcf	101.20%		-0.19%
6	445.gobmk	99.49%		-0.85%
7	456.hmmer	88.80%		0.41%
8	458.sjeng	105.14%		0.09%
9	462.libquantum	92.05%		-1.04%
10	464.h264ref	103.75%		0.82%
11	471.omnetpp	100.65%		-0.35%
12	473.astar	101.39%		0.18%
13	483.xalancbmk	104.86%		0.00%
14	geomean/GHz	0.50%	Total size	-0.31%

*not official data

Challenges in implementation

- LLVM's vector type system was not designed to handle RVV in a nice way

  `<[vscale] x N x i16>` - `[scalable]` vector of `N` elements, where is `N` compile-time known integer constant which is a power of 2.

  Special encoding is required to encode fractional LMUL as `<[vscale] x 1/8 x i16>` is not possible to have

- Loop Vectorizer was not designed to emit VL-dependent vector code
- Loop Vectorizer's cost model and next optimizations were not designed to handle VL that can change on a next vector iteration

Challenges in implementation (cont.)

- Absence of terminology to express RVV VLA. As a result, most of existing pragmas, options are irrelevant to RVV VLA. For example:

  `#pragma clang loop vectorize_width(2)`

  - does it mean VL = 2 (VLS) ?
  - does it mean LMUL=2, if so, how to represent fractional LMUL?
  - does it mean VLEN ?
  - how to specify SEW ?

  `#pragma clang loop vectorize_width(2, scalable)`

  - does it mean LMUL=2, if so, how to represent fractional LMUL?
  - does it refer to LLVM's encoding `<[vscale] x 2>` of LMUL ?
  - how to specify SEW ?

Glossary

- Primary RVL - a runtime VL which is used to increment the loop index
- Primary RVV-pair: a (LMUL, SEW) - pair that is used to compute primary RVL
- Primary LMUL - a LMUL part of the primary RVV-pair
- Primary SEW - a SEW part of the primary RVV-pair

```
void foo(const int32_t N, const int8_t *a, float *b) {
   for (int32_t i = 0; i < N; ++i) {
       b[i] = a[i] * a[i];
   }
}
```

```
        sub     a4, a0, a3
        vsetvli a4, a4, e32, m1, ta, mu
        add     a5, a1, a3
        vle8.v  v8, (a5)
        vsetvli zero, a4, e16, mf2, ta, ma
        vsext.vf2       v9, v8
        vwmul.vv        v8, v9, v9
        vsetvli zero, zero, e32, m1, ta, ma
        vfcvt.f.x.v     v8, v8
        slli    a5, a3, 2
        add     a5, a5, a2
        add     a3, a3, a4
        vse32.v v8, (a5)
        bne     a3, a0, .LBB0_3
```

```
<source>:3:4: remark: vectorized loop ((lmul, type): (1, float)) [-Rpass=loop-vectorize]

<source>:4:4: remark: vectorized loop (vectorization width: vscale x 2, interleaved
count: 2) [-Rpass=loop-vectorize]
```

RVV VLA vectorization best practices

- Use `-Rpass=loop-vectorize -Rpass-missed=loop-vectorize -Rpass-analysis=loop-vectorize` to see which loops were vectorized, not vectorized due to heuristics or due to some unhandled dependencies.
- Compiler currently limits primary LMUL to 1. Use `-mllvm -vector-primary-lmul-max=N (N=0,1,2)` to set LMUL to a compilation unit if performance is better with higher LMUL [Cost Model tuning is in progress]
  - **Don't rely on the option entirely, behavior might change**
  - The work on exposing user-convenient pragma is in progress [1]
- Don't use `#pragma clang loop vectorize_width(VL) interleave_count(IF)` as it enables VLS/SVE VLA loop vectorization
- Due to current limitations, loops that require scalarization (for example, math functions) cannot be vectorized [Enablement is in progress]
  - Temporary workaround is to use `-fno-use-vla-vectorizer`. This will enable VLS loop vectorization and math function scalarization

RVV VLA vectorization best practices (cont)

- Use `#pragma clang loop vectorize(disable)` if
  - performance degrades [Cost Model tuning is in progress]
  - runtime issue is encountered
- Use `#pragma clang loop unroll(disable)` for loops with known and low trip count to avoid complete unrolling
- For floating-point reductions, make sure that floating-point reassociation is enabled within the loop
  - Use `-ffast-math` option
  - Use `#pragma clang fp reassociate(on)` in the loop for fine-grained control
- Use `#pragma clang loop vectorize(assume_safety)` to allow vectorization of `a[b[i]] += …` statements if it's known that `b[i]` won't contain equal values in the vector

SLP vectorization

The idea of SLP is to enable vectorization for straight-line code, where the loop vectorizer cannot be applied

```
void foo_4(const int8_t *a, float *b) {
   b[0] = a[0] * a[0];
   b[1] = a[1] * a[1];
   b[2] = a[2] * a[2];
   b[3] = a[3] * a[3];
}
```

```
void foo_4(const int8_t *a, float *b) {
   b[0:3] = a[0:3] * a[0:3];
}
```

SLP's limitations

- Works on a single basic-block. That is, `b[0:3]`

```
void foo_4(const int8_t *a, float *b) {
   b[0] = a[0] * a[0];
   b[1] = a[1] * a[1];
   if (b[5] > 0) {
     return;
   }
   b[2] = a[2] * a[2];
   b[3] = a[3] * a[3];
}
```

won't be vectorized

- SLP currently handles #statements = 2^n, which doesn't utilize RVV's strength [Improve is in progress]
  - if opportunity comes from complete unrolling, for now use `#pragma clang loop unroll(disable)`
- Does not support partial tree vectorization: either entire tree is profitable or nothing.

SLP best practices

- Use `-Rpass=slp-vectorizer -Rpass-missed=slp-vectorizer -Rpass-analysis=slp-vectorizer` to see which code was vectorized, not vectorized due to heuristics or due to some unhandled dependencies.
- If you suspect SLP causes regressions, use [Cost Model tuning is in progress]
  - Play with `-slp-threshold` to disable expensive vectorization
  - `-fno-slp-vectorize` to disable SLP completely

Summary

- Many technical challenges in LLVM's vectorizers for RVV that are not yet solved
- A lot of non-technical things in LLVM were done for VLS vectorization
  pragmas, options, terminology etc.

We are eager to improve RVV support in LLVM's vectorizers and looking forward to productive discussions to do that!



Thank you

SIFIVE.COM

©2022 SiFive, Inc. All rights reserved. All trademarks referenced herein belong to their respective companies.
This presentation is intended for informational purposes only and does not form any type of warranty.

Certain information in this presentation may outline SiFive's general product direction. The presentation shall not serve to amend or affect the rights or obligations of SiFive or its licensees under any license or service agreement or documentation relating to any SiFive product. The development, release, and timing of any products, features, and functionality remains at SiFive's sole discretion.